

Reviewer Pack — Kronecker Coefficient Asymmetry in Symmetric Tensor Decompos...

Entry c2da2c902c14 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 22:58:29 UTC

Kronecker Coefficient Asymmetry in Symmetric Tensor Decompositions for Permanent vs Determinant

Entry ID: c2da2c902c14 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 22:58:29 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Boolean Circuit Complexity of Permanent

Statement:

For all $n \geq 2$, the Kronecker coefficient $g(\lambda, \mu, \nu)$ for the decomposition of $\text{Sym}^n(V) \otimes \text{Sym}^n(V)$ into irreducible representations is strictly greater for the permanent polynomial than for the determinant polynomial when λ, μ, ν are Young diagrams with at most n rows and columns.

Rationale (proposer's reasoning):

Kronecker coefficients encode the multiplicity of irreducible representations in symmetric tensor products, which are algebraic analogs of matrix multiplication. The permanent's higher symmetry compared to the determinant may manifest as asymmetric Kronecker coefficients, potentially revealing structural differences that could imply circuit complexity gaps.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5c702ef57a6b683c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(shape):
    numerator = factorial(sum(shape))
    denominator = 1
    for row in shape:
        for j in range(len(row)):
            denominator *= (row[j] + len(row) - j)
    return numerator / denominator

def kronecker_coefficient(lambda_, mu, nu, n):
    if len(lambda_) != n or len(mu) != n or len(nu) != n:
        raise ValueError("Dominant weight triples must have exactly n rows/columns")

    lambda_sum = sum(lambda_)
    mu_sum = sum(mu)
    nu_sum = sum(nu)

    if lambda_sum + mu_sum != nu_sum:
        return 0

    numerator = hook_length_formula(lambda_) * hook_length_formula(mu) * hook_length_formula(nu)
    denominator = factorial(n) ** 3
    for i in range(n):
        for j in range(n):
            for k in range(n):
                if lambda_[i] + mu[j] - nu[k] >= 0:
                    numerator /= (factorial(lambda_[i]) * factorial(mu[j]) * factorial(nu[k]))
```

```

    return numerator / denominator

def permanent(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    else:
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1) ** j) * matrix[0][j] * permanent(submatrix)
        return det

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    else:
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1) ** j) * matrix[0][j] * determinant(submatrix)
        return det

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    permanent_coeff_sum = 0
    determinant_coeff_sum = 0

    for n in n_values:
        for _ in range(18): # Aim for at least 30 instances per seed
            matrix = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

            permanent_value = permanent(matrix)
            determinant_value = determinant(matrix)

            lambda_ = [n]
            mu = [n]
            nu = [n]

            permanent_coeff = kronecker_coefficient(lambda_, mu, nu, n)
            determinant_coeff = kronecker_coefficient(lambda_, mu, nu, n)

            permanent_coeff_sum += permanent_coeff
            determinant_coeff_sum += determinant_coeff

            instances_tested += 1

    mean_perm_coeff = permanent_coeff_sum / instances_tested
    mean_det_coeff = determinant_coeff_sum / instances_tested

    conjecture_holds = mean_perm_coeff > mean_det_coeff

```

```

counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Kronecker Coefficient Asymmetry",
    "metric_value": mean_perm_coeff - mean_det_coeff,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48911ad2.py", line 125, in <module>
    result = run_trial(seed)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48911ad2.py", line 96, in run_trial
    permanent_coeff = kronecker_coefficient(lambda_, mu, nu, n)
                    ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48911ad2.py", line 36, in kronecker_coefficient
    raise ValueError("Dominant weight triples must have exactly n rows/columns")

```

ValueError: Dominant weight triples must have exactly n rows/columns

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid Young diagram dimensions; cannot evaluate conjecture's validity | next: Fix Young diagram row/column constraints in kronecker_coefficient implementation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107230
2	propose	ollama_remote	qwen3:8b	0	0	63612
3	preregistration	ollama_remote	qwen3:8b	0	0	19941
4	novelty	ollama_remote	qwen3:8b	0	0	16521
5	novelty	ollama_remote	qwen3:8b	0	0	14297
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13688
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13610
8	judge	ollama_remote	qwen3:8b	0	0	11983

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 260882 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/c2da2c902c14.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c2da2c902c14.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c2da2c902c14.tar.gz (if generated)